COMP 416
Analysis of Transport Layer Protocols through Wireshark
Saliha Zülal Demirtaş 79747

Part 1. TCP Experiments

Part 1.1 What is the sequence number of the TCP segment containing the header of the HTTP POST request?

I used *http.request.method == POST* display filter to query HTTP Post request:

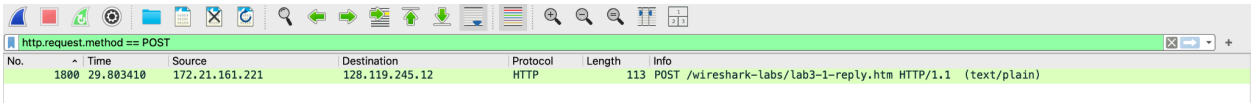


Figure 1: Filtered Wireshark Trace Output

The raw sequence number chosen randomly by the sender is 3919220751. Wireshark uses relative sequence numbers to start the sequence numbers from 0. The relative sequence number, generated by Wireshark, is 152940.

```
> Frame 733: Packet, 113 bytes on wire (904 bits), 113 bytes captured (904 bits) on interface en0, id 0
> Ethernet II, Src: Apple_b0:53:74 (c8:89:f3:b0:53:74), Dst: CheckPointSo_48:ad:0a (00:1c:7f:48:ad:0a)
> Internet Protocol Version 4, Src: 172.21.165.186, Dst: 128.119.245.12
v Transmission Control Protocol, Src Port: 53225, Dst Port: 80, Seq: 152893, Ack: 1, Len: 47
    Source Port: 53225
    Destination Port: 80
    [Stream index: 20]
    [Stream Packet Number: 150]
  > [Conversation completeness: Complete, WITH_DATA (31)]
    [TCP Segment Len: 47]
    Sequence Number: 152893    (relative sequence number)
    Sequence Number (raw): 3919220751
    [Next Sequence Number: 152940    (relative sequence number)]
    Acknowledgment Number: 1    (relative ack number)
    Acknowledgment number (raw): 415024842
    1000 .... = Header Length: 32 bytes (8)
  > Flags: 0x018 (PSH, ACK)
    Window: 2051
    [Calculated window size: 131264]
    [Window size scaling factor: 64]
    Checksum: 0xb597 [unverified]
    [Checksum Status: Unverified]
    Urgent Pointer: 0
```

Figure 2: TCP Header of the HTTP POST Request

Part 1.2 Using Wireshark, plot the Time-Sequence graph for the TCP stream, and discuss the different phases of TCP using the graph.

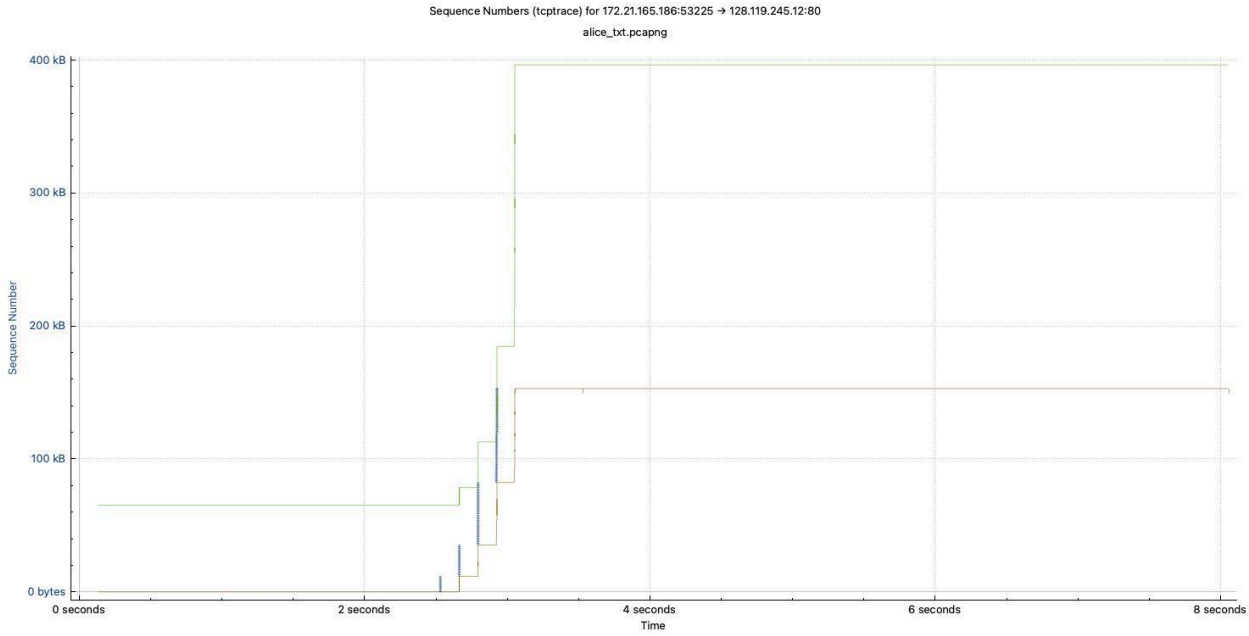


Figure 3: TCP Time-Sequence Graph

Using Follow → TCP Stream → Statistics → TCP Stream Graph → Time-Sequence (tcptrace), I plotted the sequence number of the TCP stream over time.

1. **Connection Establishment:** The TCP connection is established first with a three-way handshake in milliseconds. Between 0 and 2.3 seconds, the line is flat because the data transmission has not started yet.
2. **Slow Start:** At around 2.3 seconds, the sender starts sending data. For congestion control, the sending rate starts slow. The congestion window is initially set to 1 MSS, and it increases by 1 MSS for each ACK, resulting in doubling MSS in each RTT. In Figure 3, it can be seen that the step height doubles each time.
3. **Congestion Avoidance:** In theory, congestion avoidance starts when cwnd reaches the slow-start-threshold. In this part, the MSS increases linearly rather than exponentially, resulting in a linear graph. However, in this capture, the transfer completes before congestion avoidance is noticeable, so the graph does not show a linear slope. This may be because the transferred file is small.

4. **Server Response:** At around the 3.0 seconds, the server responds, resulting in a sudden jump in sequence number because large data is sent .
5. **End of Transmission:** After ~3 seconds, the graph flattens as data transmission is completed.

Part 1.3 Compare the sequence and acknowledgment number fields of two different packets of your choice. One should be a data packet, while the other should be a pure acknowledgment packet.

I selected packet 758 from the HTTP response and packet 580 from pure ACK packets.

a. **Data Packet (**Packet 758 - HTTP POST response):

- **Relative sequence number:** 1
- **Next sequence number:** 781
- **Acknowledgment number:** 152940

  Because the payload is 780 bytes, the next sequence number becomes 781.

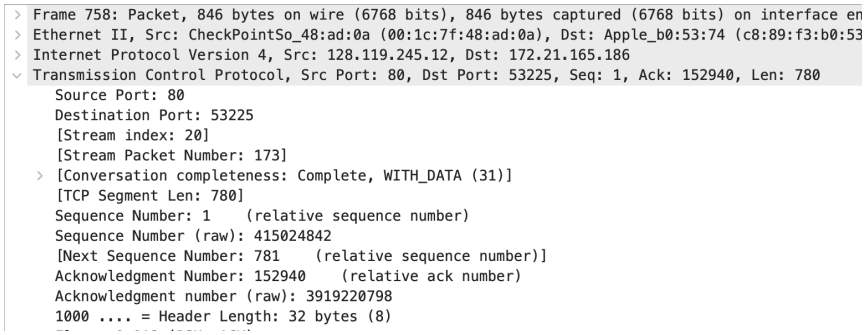


Figure 4: TCP Header of the Data Packet

b. **Pure Acknowledgment Packet** (Packet 580)**:**

- **Relative sequence number:** 1
- **Next sequence number:** 1
- **Acknowledgment Number:** 10523

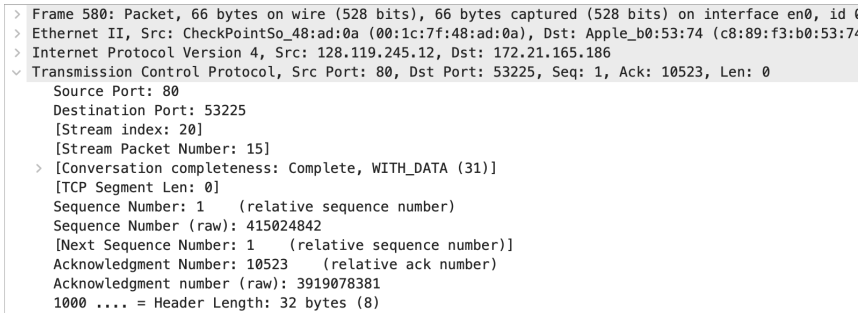


Figure 5: TCP Header of the Pure Acknowledgment Packet

c. Comparison:

Sequence numbers:

- **Data Packet:** The next sequence number increases from 1 to 781, because the payload length is 780 bytes.
- **Pure Acknowledgement Packet:** The next sequence number remains the same because the packet carries no data.

Acknowledgement numbers:

- **Data Packet:** It acknowledges the receipt of up to 152,939 bytes. So, the acknowledgement number is 152,940 as it is set to the next expected byte.
- **Pure Acknowledgement Packet:** It acknowledges the receipt of up to 10,522 bytes. So, the acknowledgement number is 10,523.
- The difference shows that the receiver processed significantly more bytes up to that data packet. The reason might be that the data transmission occurred at the packet 733, which is later than packet 580.

Part 1.4 Report the total number of packets sent from your machine during the TCP stream. A screenshot should show the applied filter and the packet count. Then, briefly discuss the factors that contributed to the number of packets.

IP source: 172.21.165.186

To count the number of packets, I applied the filter *ip.src= 172.21.165.186 && tcp.*

- Wireshark shows that 130 TCP packets were sent from my machine during the TCP stream.

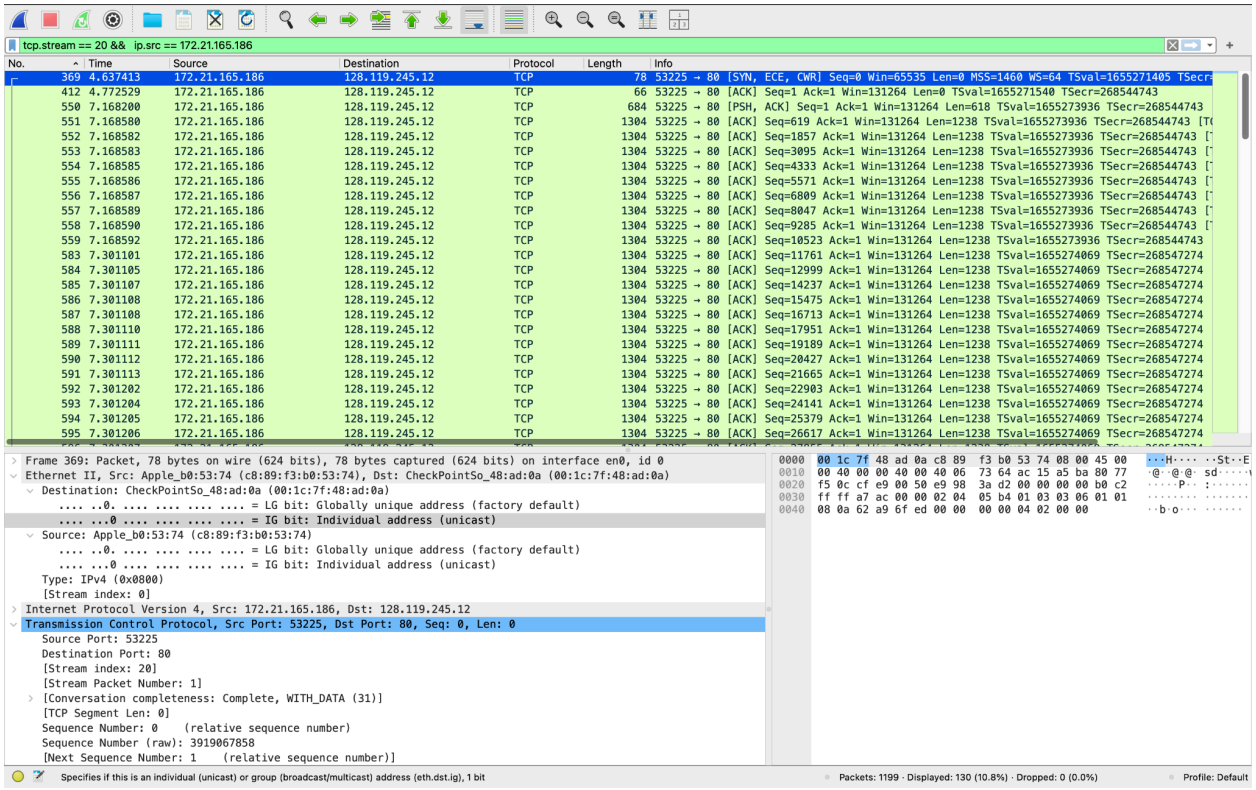


Figure 6: Packet Count Screenshot, Filter Applied

The packets include the following:

1. **The three-way handshake:** The connection is established by SYN → SYN/ACK → ACK packets. The sender first sends a TCP SYN message and the receiver acknowledges it by sending a TCP SYNACK message. Finally the sender sends ACK for SYNACK, indicating that connection is established.
2. **HTTP POST Request Packets:** After the handshake, the client sends the HTTP POST request
3. **Data Packets:** These packets carry the payload, Alice.txt.
4. **Acknowledgement Packets:** The client sends ACK packets during the transfer to confirm the receipt of server's TCP segments, due to HTTP POST Response.
5. **TCP Keep Alive:** At the end of transmission, TCP Keep Alive packet appears in the capture. This allows the sender to check whether the receiver is still alive.

Part 1.5 Are there any retransmitted segments in the trace file? What did you check for (in the trace) in order to answer this question?

There are no retransmitted segments in the trace. First I applied *tcp.analysis.retransmission* filter, and it returned no packets. Then, I checked for duplicate ACKs and they were not found. ACK numbers increased consistently. Thus, I concluded that there were no retransmitted segments.

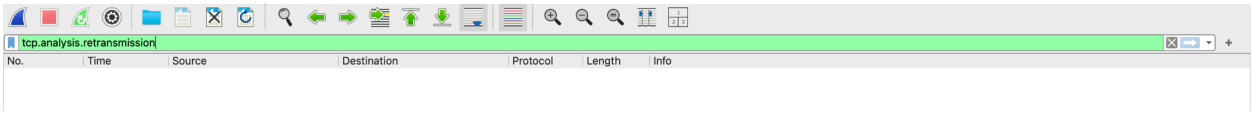


Figure 7: Wireshark Output Showing No Retransmissions

Part 2. UDP and QUIC Experiments

Part 2.1 How can you use Wireshark to verify that the UDP header is always 8 bytes, while the TCP header size varies from packet to packet? Which Wireshark fields show this?

UDP headers consist of Source Port, Destination Port, Length, and Checksum, which each one is 2 bytes. When we click these fields in Wireshark, it shows that each field is 2 bytes, adding up to a total header size of 8 bytes. Additionally, we can also confirm it by subtracting the UDP payload length from the total UDP length. When the trace is filtered with *udp* filter and multiple packets are analyzed, it can be seen that the UDP header length is always 8 bytes.

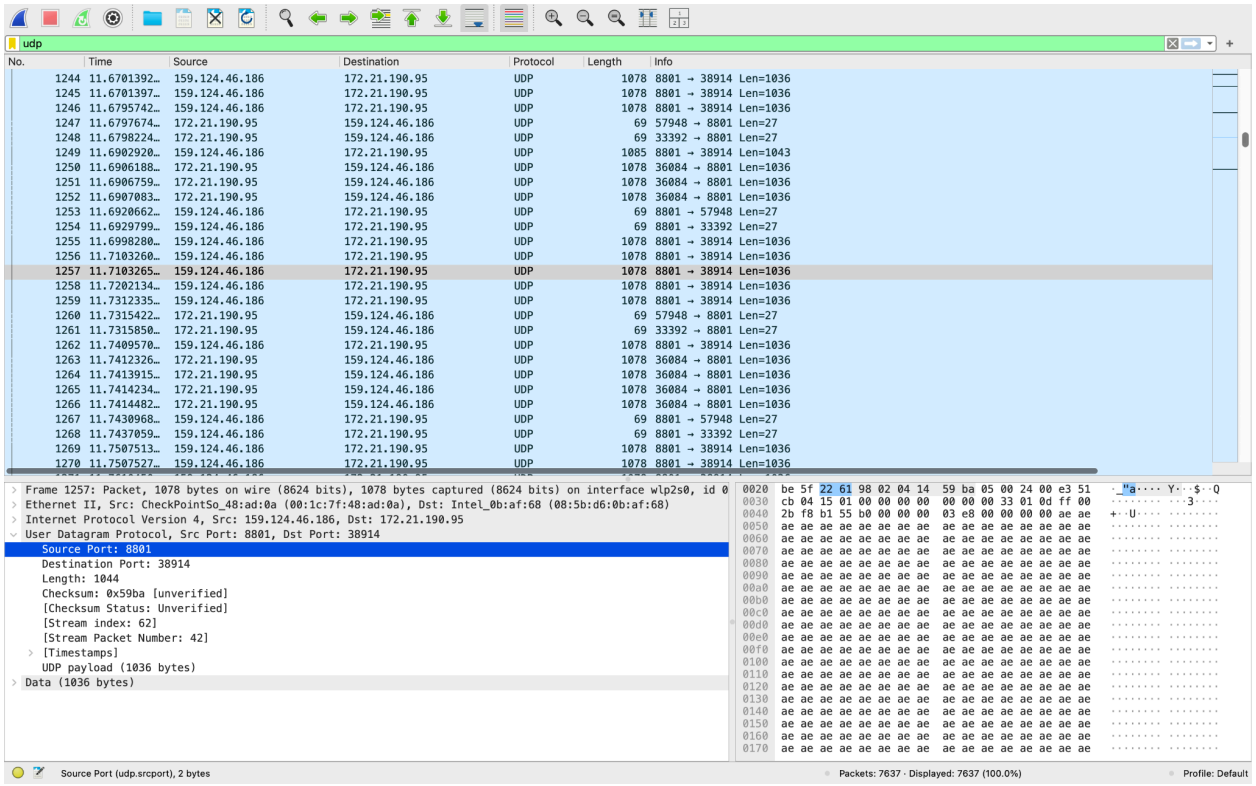


Figure 8: UDP Header Content

Part 2.2 Find an instance where it is clear a UDP packet failed to reach its destination. Provide screenshots, and discuss how you identified this packet, citing any ICMP error messages or application-level retransmissions that occurred. Can we be certain that the remaining UDP packets in the trace were successfully delivered?

I identified the failed UDP packet by checking the Info column for error messages. The error also can be found directly by applying the filter *udp && icmp*, which shows ICMP error responses of UDP packets. In this trace, the ICMP message is “Destination unreachable (Port unreachable)”.

We cannot be certain that the remaining UDP packages were delivered successfully. UDP is an unreliable protocol and provides no acknowledgements or retransmissions. Unlike TCP, it does not establish a connection or maintain state about packet delivery. Hence, the absence of an ICMP error does not guarantee that the error did not occur.

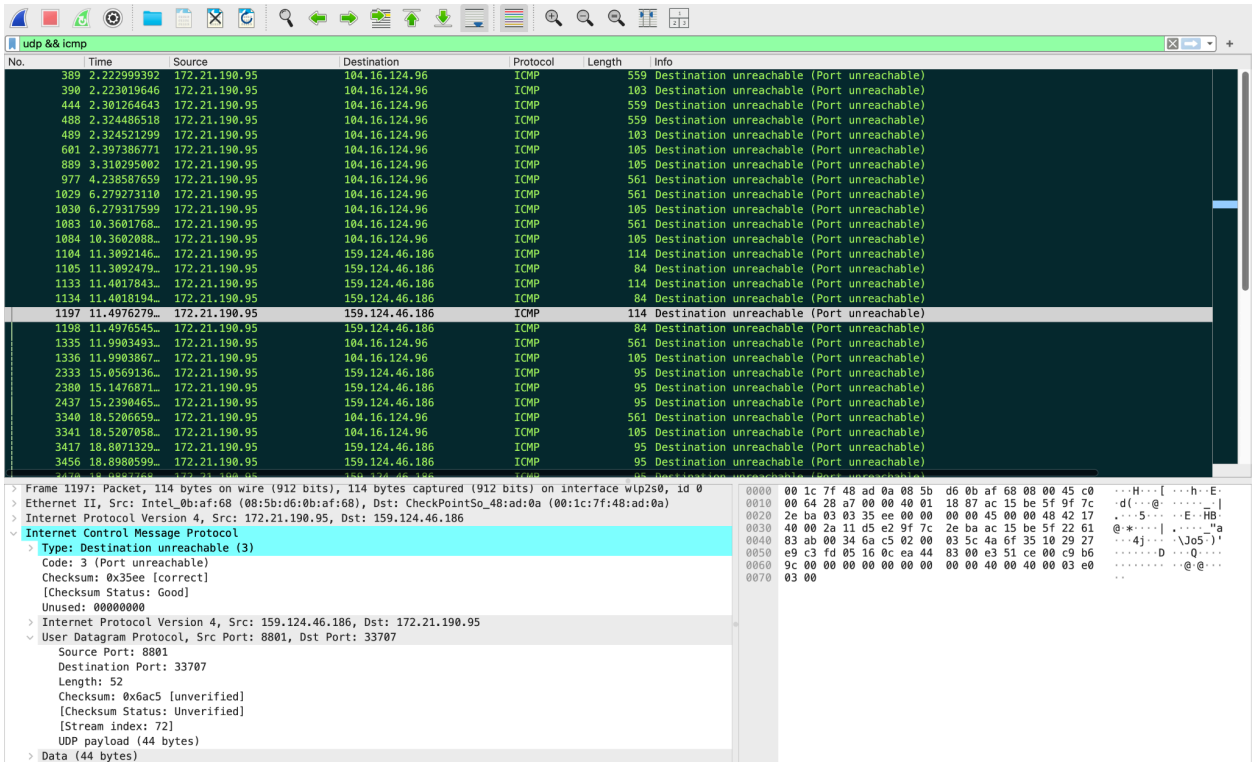
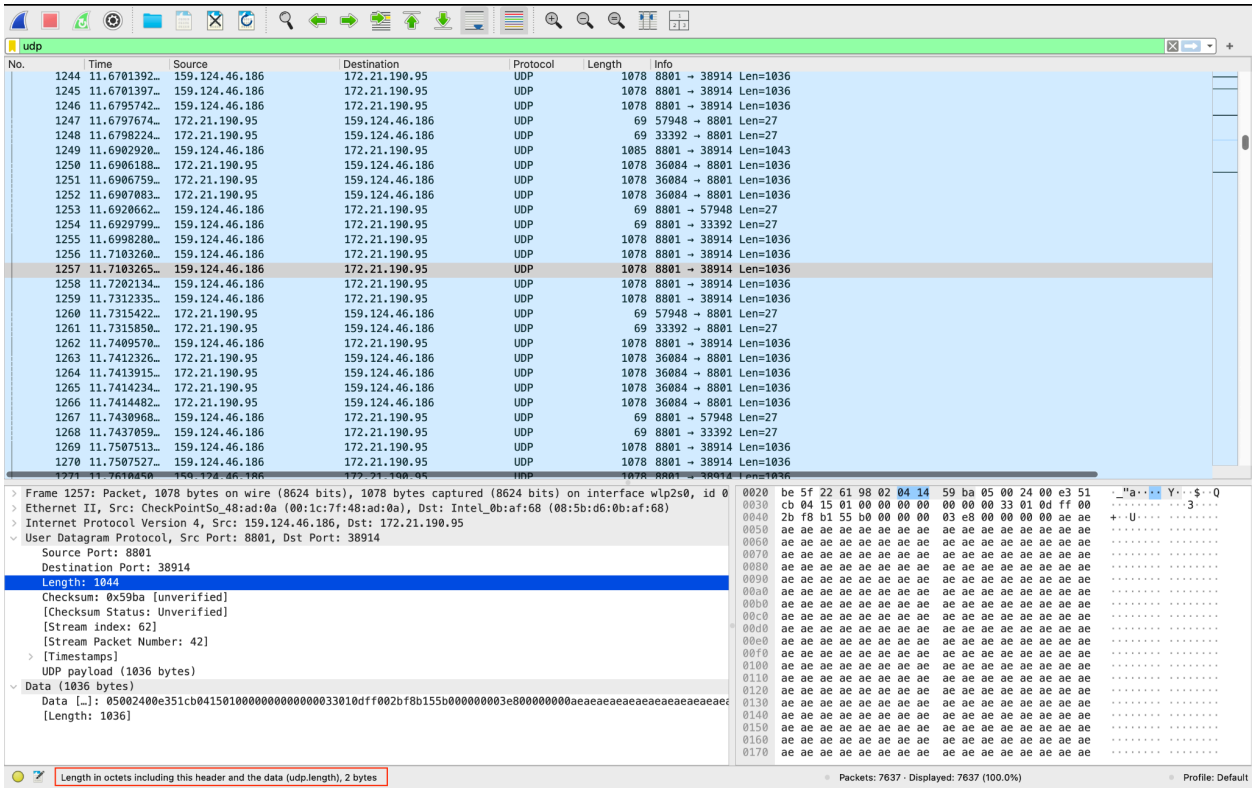


Figure 9: Wireshark output, Packets with "Destination Unreachable" Error

Part 2.3 By consulting the displayed information in Wireshark, deduce the maximum possible value for the length field of a UDP packet. Explain your answer using a screenshot from the trace.

In the trace, Wireshark shows that the field is 2 bytes, which corresponds to a 16-bit integer. So the maximum possible value for the UDP length field is 2^16 - 1 = 65,535.


udp
No. Time Source Destination Protocol Length Info
1244 11.6701392… 159.124.46.186 172.21.190.95 UDP 1078 8801 → 38914 Len=1036
1245 11.6701397… 159.124.46.186 172.21.190.95 UDP 1078 8801 → 38914 Len=1036
1246 11.6795742… 159.124.46.186 172.21.190.95 UDP 1078 8801 → 38914 Len=1036
1247 11.6797674… 172.21.190.95 159.124.46.186 UDP 69 57948 → 8801 Len=27
1248 11.6798224… 172.21.190.95 159.124.46.186 UDP 69 33392 → 8801 Len=27
1249 11.6902920… 159.124.46.186 172.21.190.95 UDP 1085 8801 → 38914 Len=1043
1250 11.6906188… 172.21.190.95 159.124.46.186 UDP 1078 36084 → 8801 Len=1036
1251 11.6906759… 172.21.190.95 159.124.46.186 UDP 1078 36084 → 8801 Len=1036
1252 11.6907083… 172.21.190.95 159.124.46.186 UDP 1078 36084 → 8801 Len=1036
1253 11.6920662… 159.124.46.186 172.21.190.95 UDP 69 8801 → 57948 Len=27
1254 11.6929799… 159.124.46.186 172.21.190.95 UDP 69 8801 → 33392 Len=27
1255 11.6998280… 159.124.46.186 172.21.190.95 UDP 1078 8801 → 38914 Len=1036
1256 11.7103260… 159.124.46.186 172.21.190.95 UDP 1078 8801 → 38914 Len=1036
1257 11.7103265… 159.124.46.186 172.21.190.95 UDP 1078 8801 → 38914 Len=1036
1258 11.7202134… 159.124.46.186 172.21.190.95 UDP 1078 8801 → 38914 Len=1036
1259 11.7312335… 159.124.46.186 172.21.190.95 UDP 1078 8801 → 38914 Len=1036
1260 11.7315422… 172.21.190.95 159.124.46.186 UDP 69 57948 → 8801 Len=27
1261 11.7315850… 172.21.190.95 159.124.46.186 UDP 69 33392 → 8801 Len=27
1262 11.7409570… 159.124.46.186 172.21.190.95 UDP 1078 8801 → 38914 Len=1036
1263 11.7412326… 172.21.190.95 159.124.46.186 UDP 1078 36084 → 8801 Len=1036
1264 11.7413915… 172.21.190.95 159.124.46.186 UDP 1078 36084 → 8801 Len=1036
1265 11.7414234… 172.21.190.95 159.124.46.186 UDP 1078 36084 → 8801 Len=1036
1266 11.7414482… 172.21.190.95 159.124.46.186 UDP 1078 36084 → 8801 Len=1036
1267 11.7430968… 159.124.46.186 172.21.190.95 UDP 69 8801 → 57948 Len=27
1268 11.7437059… 159.124.46.186 172.21.190.95 UDP 69 8801 → 33392 Len=27
1269 11.7507513… 159.124.46.186 172.21.190.95 UDP 1078 8801 → 38914 Len=1036
1270 11.7507527… 159.124.46.186 172.21.190.95 UDP 1078 8801 → 38914 Len=1036
Frame 1257: Packet, 1078 bytes on wire (8624 bits), 1078 bytes captured (8624 bits) on interface wlp2s0, id 0
Ethernet II, Src: CheckPointSo_48:ad:0a (00:1c:7f:48:ad:0a), Dst: Intel_0b:af:68 (08:5b:d6:0b:af:68)
Internet Protocol Version 4, Src: 159.124.46.186, Dst: 172.21.190.95
User Datagram Protocol, Src Port: 8801, Dst Port: 38914
Source Port: 8801
Destination Port: 38914
Length: 1044
Checksum: 0x59ba [unverified]
[Checksum Status: Unverified]
[Stream index: 62]
[Stream Packet Number: 42]
[Timestamps]
UDP payload (1036 bytes)
Data (1036 bytes)
Data […]: 05002400e351cb0415010000000000000033010dff002bf8b155b00000003e800000000aeaeaea

QUIC establishes a connection, by sending Initial packets, containing TLS 1.3 handshake messages inside CRYPTO frames. The client first sends an Initial packet with ClientHello.

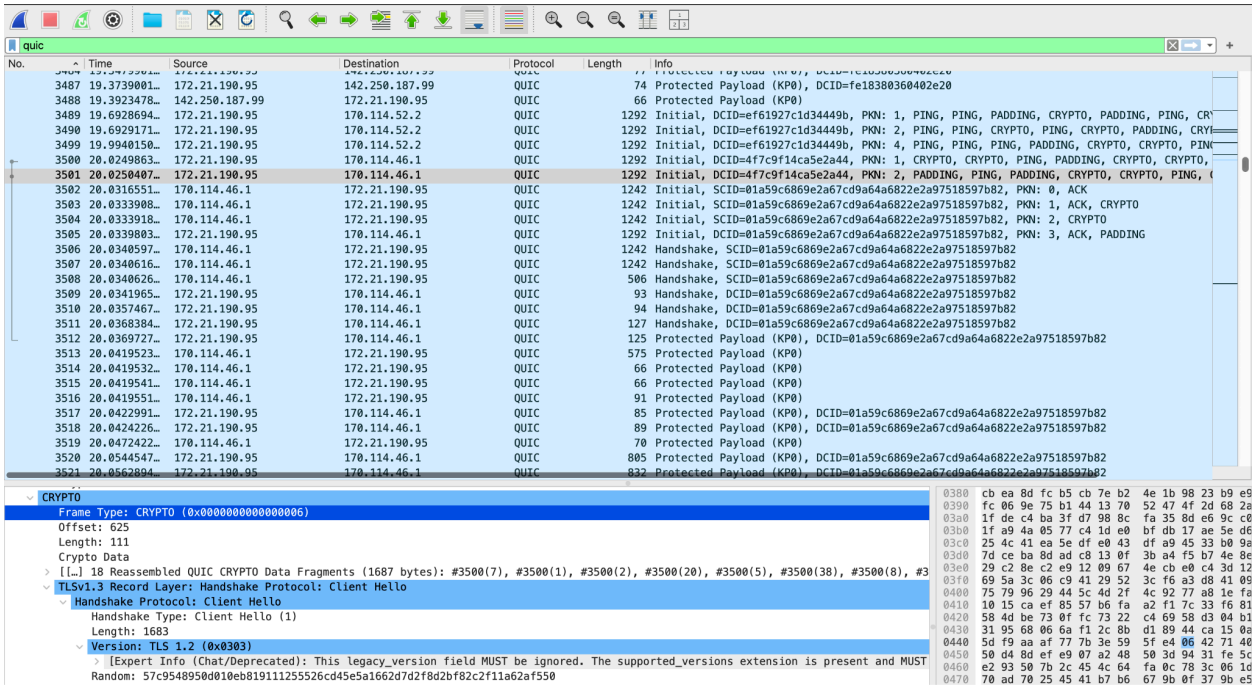


Figure 11: QUIC Initial Packet Containing ClientHello Message

The server replies with Initial packets that include ACK and ServerHello, followed by server Handshake packets. Then, the client also sends a Handshake packet, as it can be seen in the trace. The QUIC packet that contains the last Handshake frame is 3510.

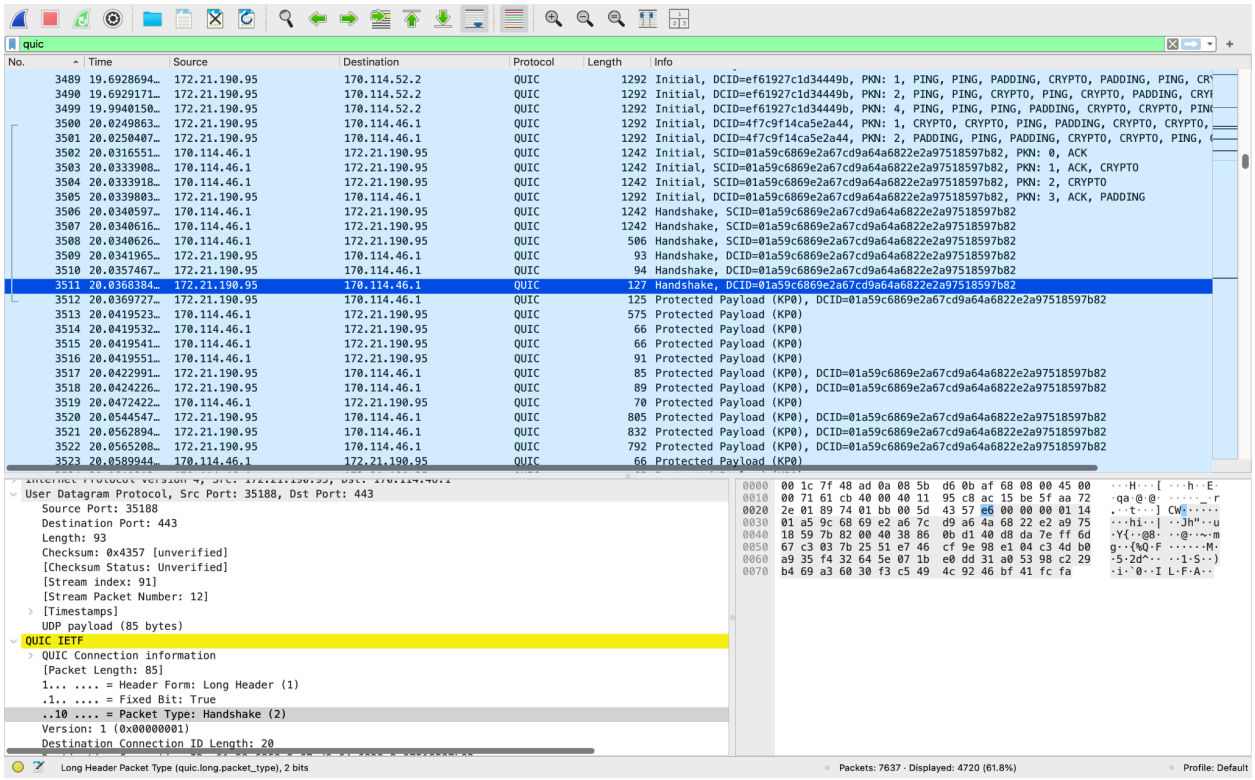


Figure 12: QUIC Server Initial and Handshake Packets

In TCP + TLS handshake, the connection is established in two separate handshakes, costing additional RTTs. First, TCP connection is established with a three-way handshake. Then, TLS handshake messages are carried in the TCP payload. The certificate check and key exchange are done in multiple RTTs. QUIC is built on UDP and it directly integrates TLS into its transport, so establishment starts immediately, hence connection setup is faster.

Part 2.6 Show a UDP packet carrying more than one QUIC frame, identify the different frame types, and discuss the possible implications of this observation. Provide screenshots of the referenced packet.

Packet 3777 contains multiple CRYPTO, PING, and PADDING frames.

1. **CRYPTO Frames**

   CRYPTO frames carry TLS1.3 handshake data. These frames contain the handshake messages, including ClientHello and ServerHello.

2. **PING Frames**

   PING Frames are used to check that connection is still live. They help trigger acknowledgments.

3. **PADDING Frames**

   PADDING frames do not contain any data. They are used to increase the size of Initial packets to at least 1200 bytes. They are used to prevent attacks.

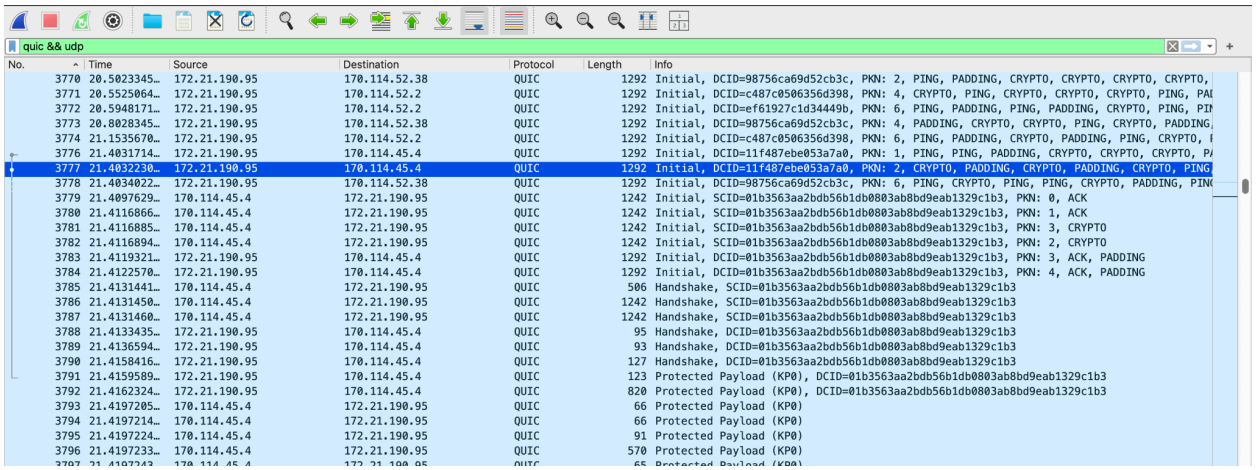


Figure 13: QUIC Packet Containing Multiple Frames

Part 3. TCP vs SSL Implementations and Analysis

Part 3.1 Connection Establishment

Describe the steps involved in establishing a TCP connection and an SSL connection. Use Wireshark screenshots in your explanation.

A TCP connection is established using the standard three-way handshake. First, the client sends a SYN packet to indicate that it wants to start a connection. The server replies with a SYN-ACK packet, confirming that it received the request. Finally, the client replies with an ACK, completing the handshake and establishing a connection. In my Wireshark, I filtered my capture by *tcp.port == 8000* filter to observe my connection. The packets 61 (SYN), 62 (SYN-ACK), and 63 (ACK) show the three-way handshake.

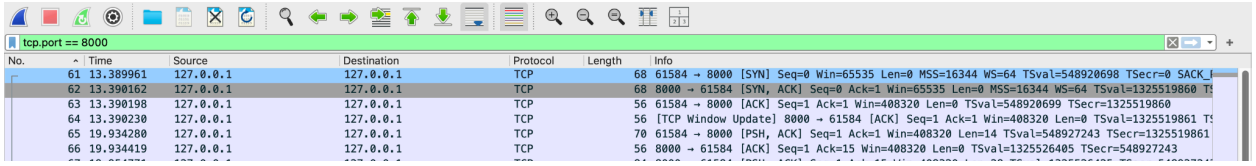


Figure 14: Wireshark Output Containing TCP Three-way Handshake

A SSL connection is established after TCP three-way handshake connection completes. It runs on top of TCP. The client sends a ClientHello, telling the server which version and options it supports. The server responds with a ServerHello, choosing the encryption settings. Then the server sends additional handshake messages, including its certificate, so the client can verify its identity. When both sides exchange key information and send their Finished messages, the handshake is complete.

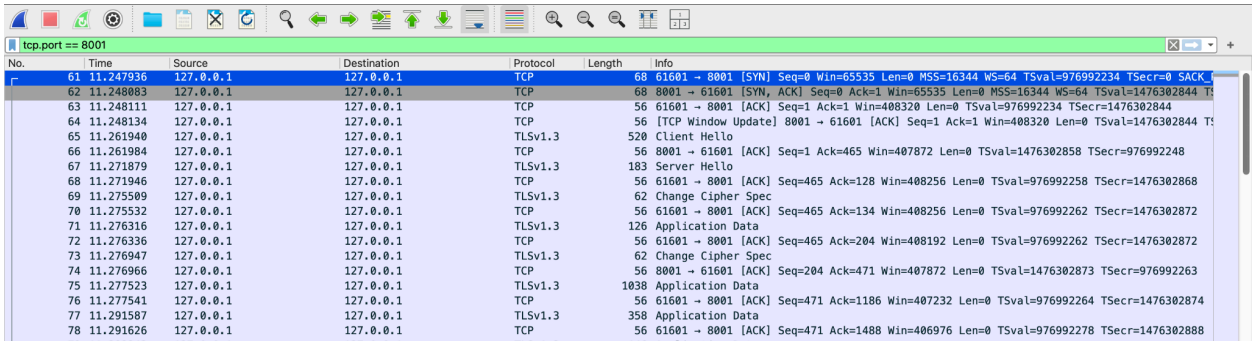


Figure 14: Wireshark Output Containing SSL Establishment

SSL Certificate Details:

```
(base) zulaldemirtas@Zulals-MacBook-Pro SensorServer % keytool -list -v -keystore keystore.jks -storepass aa12345
Keystore type: PKCS12
Keystore provider: SUN

Your keystore contains 1 entry

Alias name: mykey
Creation date: Dec 9, 2025
Entry type: PrivateKeyEntry
Certificate chain length: 1
Certificate[1]:
Owner: CN=Saliha Zülal Demirtas, OU=COMP416, O=Koç University, L=Istanbul, ST=Turkey, C=TR
Issuer: CN=Saliha Zülal Demirtas, OU=COMP416, O=Koç University, L=Istanbul, ST=Turkey, C=TR
Serial number: 3dc879e7838c3b2d
Valid from: Tue Dec 09 02:46:10 TRT 2025 until: Mon Mar 09 02:46:10 TRT 2026
Certificate fingerprints:
         SHA1: 2D:67:E5:8A:51:4E:3C:90:3A:6E:23:73:99:EC:21:7F:55:F9:47:58
         SHA256: 83:DE:8A:18:44:2E:25:E1:9F:C4:16:02:C6:45:CC:CA:DE:C5:9E:09:9C:48:C2:F6:FD:97:3A:09:63:41:66:D1
Signature algorithm name: SHA256withRSA
Subject Public Key Algorithm: 2048-bit RSA key
Version: 3

Extensions:

#1: ObjectId: 2.5.29.14 Criticality=false
SubjectKeyIdentifier [
KeyIdentifier [
0000: 86 4B 94 14 6E D7 C7 09   7C 3A 63 92 5F D2 F4 78  .K..n....:c._..x
0010: D2 BA CE B6                                        ....
]
]
```

Figure 15: SSL Certificate Details

Part 3.2 Security Differences

Explain what types of security protections SSL provides during streaming that plain TCP does not. Show the captured sensor data in TCP and SSL from Wireshark.

TCP only delivers the data reliably, meaning that no data is lost. On the other hand, SSL provides additional security. After the SSL handshake, all data is encrypted. Additionally, during the handshake, the client requires a certificate to verify authenticity. TCP does not do that.

In TCP, data can be read in plain text directly.

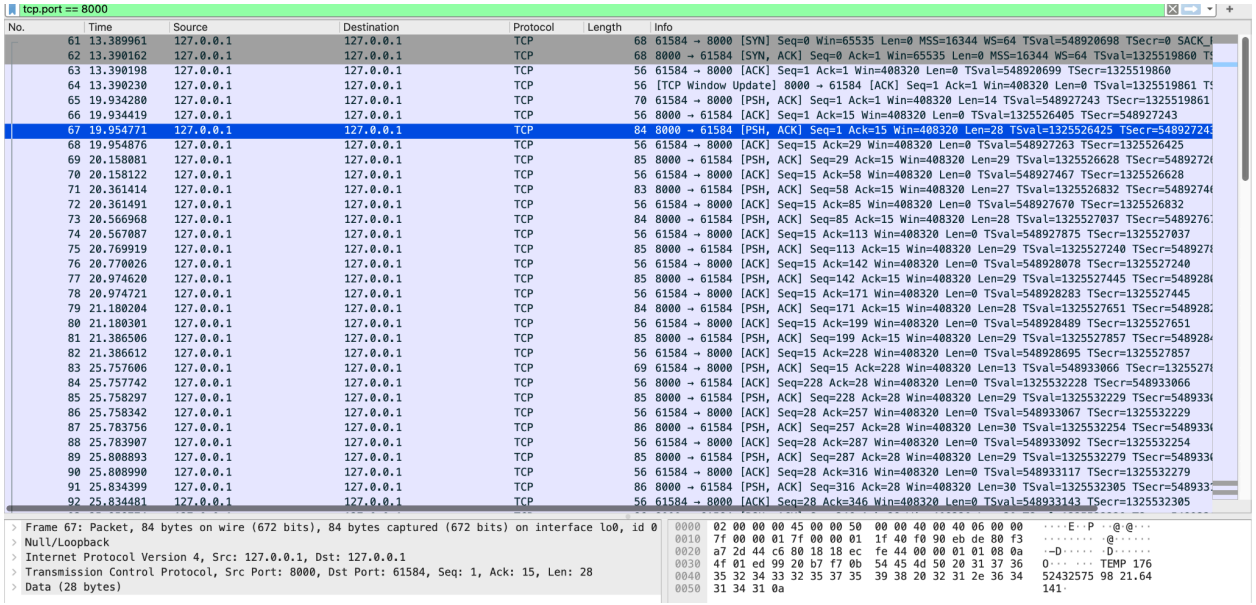


Figure 16: TCP Sensor Data Showing Plaintext Payload

In SSL capture, data is encrypted and payload is unreadable.

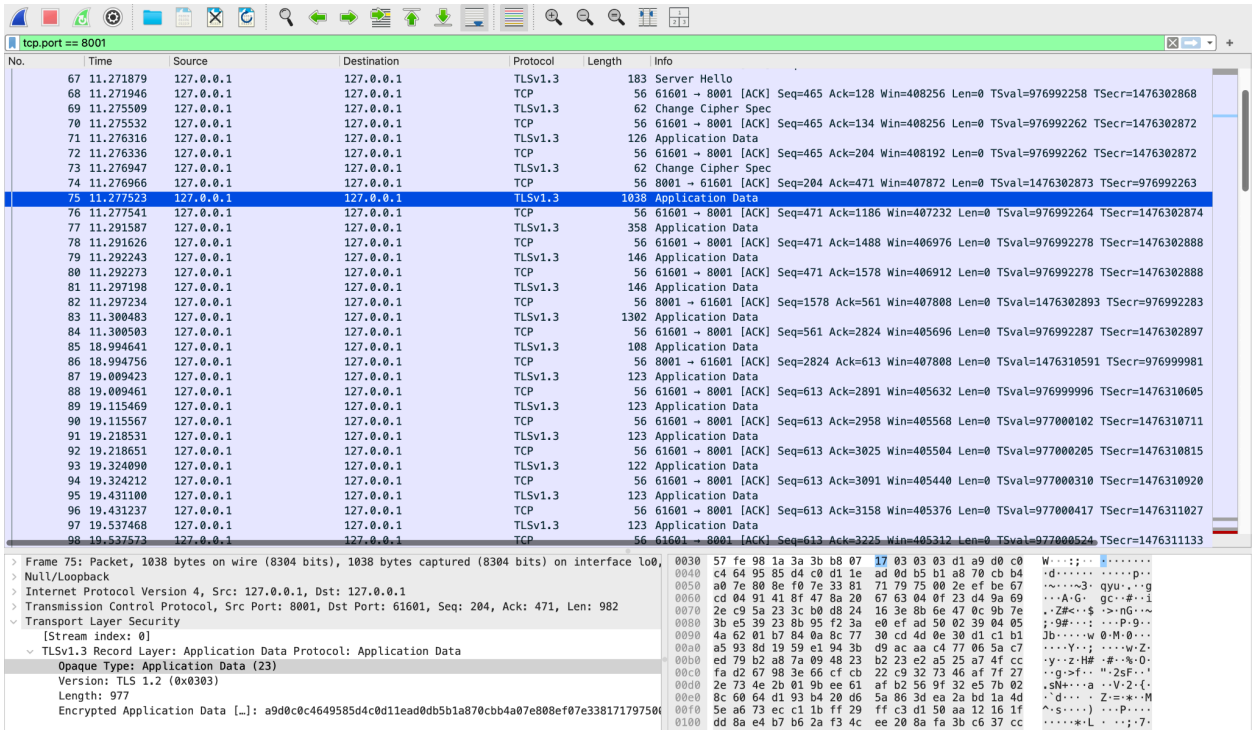


Figure 17: SSL Sensor Data Showing Encrypted Data

Part 3.3 Performance Comparison

I incorporated benchmark mode into the client mode selection. In benchmark mode,the client performs 20 runs in total, 10 using TCP and 10 using SSL. A new connection is established for each run and timestamps of the messages are recorded.

When compared, the SSL connection showed a faster first-message arrival (1 ms) compared to TCP (3 ms). Even though it is expected SSL connection to be slower due to additional TLS handshake, this result may be due to that both the server and client run on localhost.

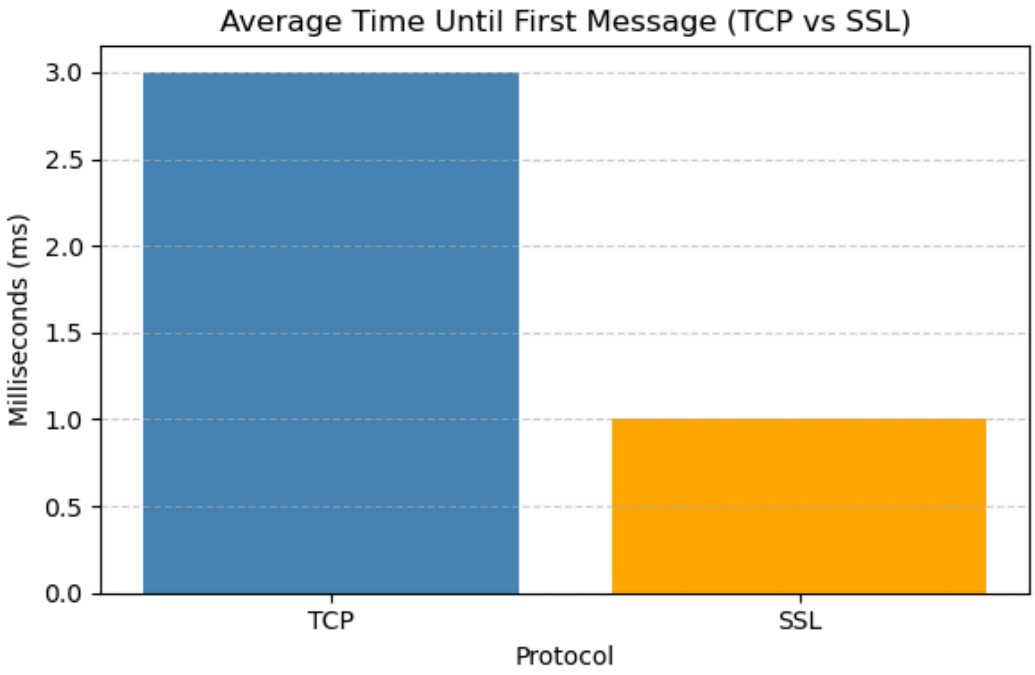


Figure 18: Average Time Until First Message (TCP vs SSL)

Both TCP and SSL showed similar average delays, TCP delay with 23 ms and SSL delay with 24 ms. This shows that encryption does not significantly affect delivery time per message.

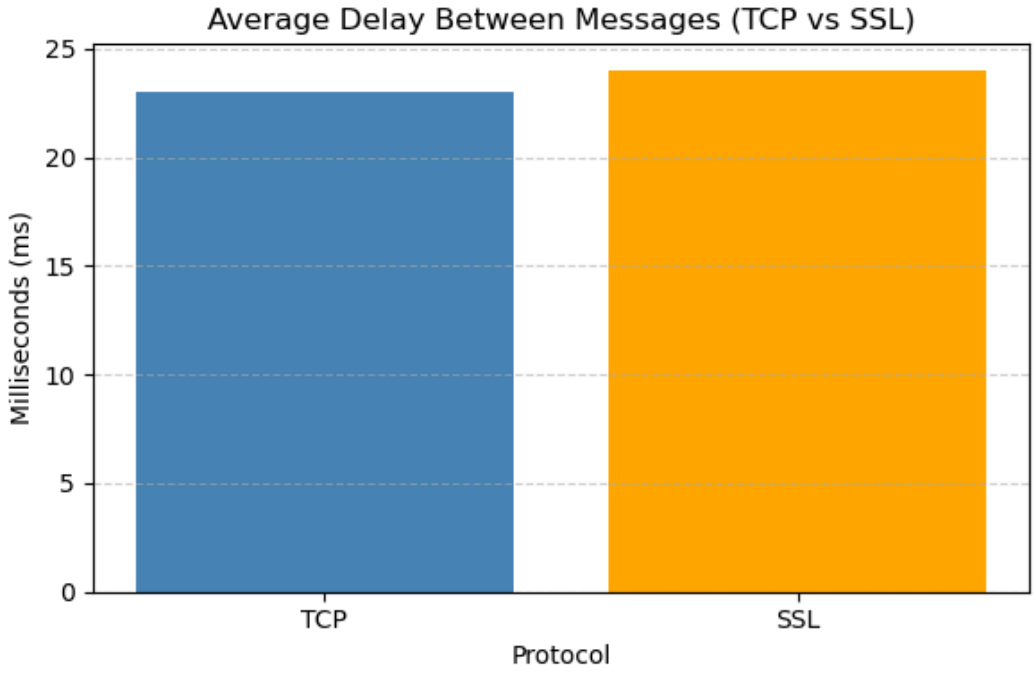


Figure 19: Average Delay Between Messages (TCP vs SSL)